



MACHINE LEARNING FOR SOFTWARE ENGINEERING

Eugenio Di Gaetano - 0349278

Agenda

01

Introduzione

02

Progettazione

03

Variabili

04

Risultati

05

Minacce alla validità

06

Link

Introduzione

Contesto

- Il processo di testing del software è una fase critica nello sviluppo, mirata a **identificare e correggere bug** nel codice. Tuttavia, testare interamente un'applicazione di grandi dimensioni è spesso impraticabile a causa di vincoli di tempo e risorse.
- I modelli di **Machine Learning (ML)** possono essere impiegati per prevedere quali sono le classi più propense a contenere bug, consentendo ai team di testing di concentrare gli sforzi sulle aree più critiche.



Introduzione

Scopo

- L'obiettivo di questo lavoro è **ottimizzare il processo di testing**, rendendolo più agevole, efficiente e meno dispendioso in termini di risorse.
- Verranno analizzati due progetti di Apache Software Foundation, nello specifico **BookKeeper e Syncope**, misurando l'accuratezza di **tre classificatori**, usando tecniche di **sampling**, classificazioni **sensibili al costo** e **feature selection**.

Progettazione

JIRA e Git

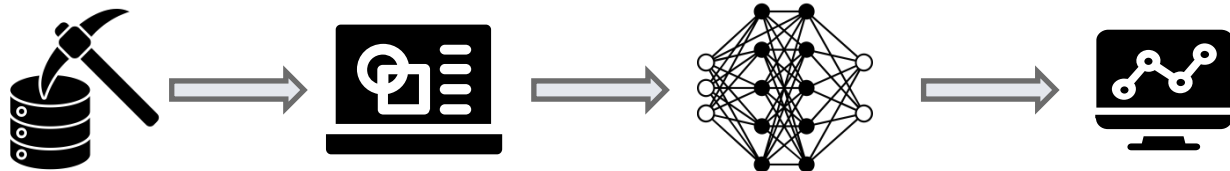
Per poter effettuare predizioni accurate su quali classi sono più suscettibili ai bug, è essenziale **raccogliere dati** da fornire ai modelli di Machine Learning.

JIRA è un software che consente il **bug tracking** e la gestione dei progetti.

Git è un sistema di **controllo di versione** distribuito.

Dati da estrarre:

- Lista delle Release
- Lista dei Ticket Bug
- Lista dei commit
- Lista delle classi modificate



Progettazione

Individuazione delle classi buggy

Per **costruire il dataset** dobbiamo identificare quali classi sono buggy, e in quali release.



Una classe è considerata buggy dalla injected version (inclusa) fino alla fixed version (esclusa)

Progettazione

Proportion

La **OV** e la **FV** sono sempre individuabili dai ticket di Jira,
ciò non è vero per l'**IV**.

Soluzione:

L'intuizione alla base di **proportion** è che la **distanza** in release tra OV e FV è **proporzionale** alla distanza tra la IV e la FV. In questo modo è possibile calcolare il valore di Proportion p per tutti quei bug di cui è nota l'IV, per poi predire l'IV in quei casi dove non è disponibile proprio grazie al valore di p.

$$p = (FV - IV) / (FV - OV)$$

$$IV = FV - (FV - OV) * p$$

Progettazione

Tecniche di Proportion

Per la stima dell'IV sono state utilizzate due diverse tecniche di propotion:

Se si hanno a disposizione almeno 5 ticket validi con **IV** si applica «**Proportion Increment**».

Calcolo p utilizzando **tutti i ticket** a disposizione fino a quel momento.

CONTRO:

Inizialmente potrei non avere abbastanza ticket.

Se si hanno a disposizione meno di 5 ticket validi si applica «**Proportion Cold Start**».

Calcolo p utilizzando **altri progetti** analoghi e lo imposto pari alla **mediana**.

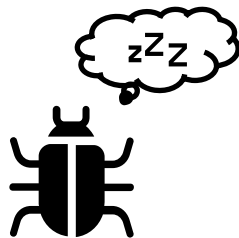
CONTRO:

Correlazione minore con il progetto attuale.

Progettazione

Snoring

Un bug, dopo essere introdotto, può rimanere “dormiente” per molto tempo; Questo determina che le ultime release siano piene di **bug dormienti** e quindi i relativi dati sulle classi buggy non siano affidabili.



Soluzione:

La soluzione attuata è quella di **scartare la seconda metà** delle release così da avere una elevata affidabilità sui dati.

Progettazione

Metriche Considerate

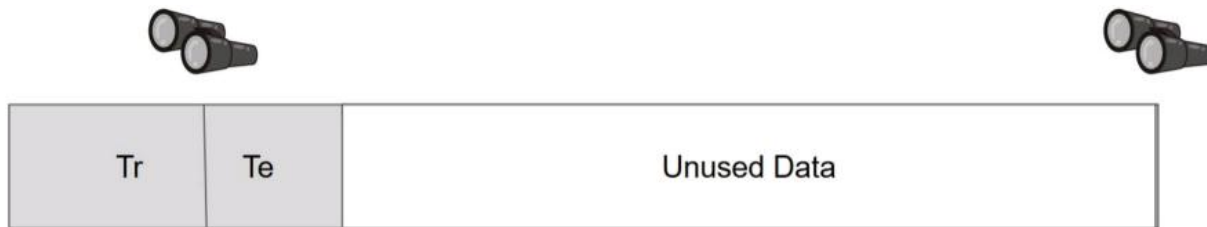
- LOC: Linee di codice
- LOC touched: Somma delle linee modificate nelle revisioni
- NFix: Numero di bug fixati
- NAuth: Numero di autori
- LOC added: Linee aggiunte nelle revisioni
- MAX LOC added: Numero massimo di linee aggiunte nelle revisioni
- Churn: Valore assoluto della differenza tra linee aggiunte e rimosse
- MAX Churn: Churn massimo nelle revisioni
- AVG Churn: Churn medio nelle revisioni
- AVG Holiday: Numero di giorni festivi tra due revisioni/ Numero di giorni totali (metrica personalizzata)

Progettazione

Training e Testing

Il dataset viene suddiviso in: training set e testing set:

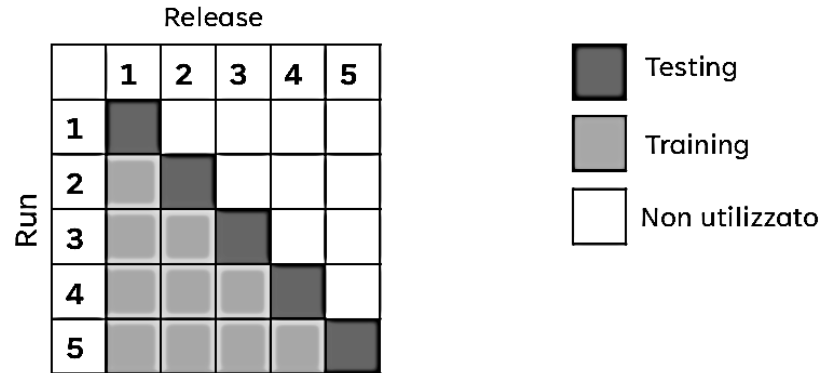
- Il training set viene utilizzato per effettuare l'addestramento dei modelli predittivi, quindi, è importante che sia il più fedele possibile al contesto reale.
- Il testing set viene utilizzato per la valutazione del modello. Deve quindi usare tutti i dati a disposizione. Non è affetto da snoring.



Progettazione

Walk Forward

Come tecnica di valutazione è stata utilizzata walk-forward in quanto è una tecnica time-series (ossia preserva l'ordine temporale), caratteristica essenziale per non utilizzare impropriamente dati del futuro per predire il passato.



Variabili

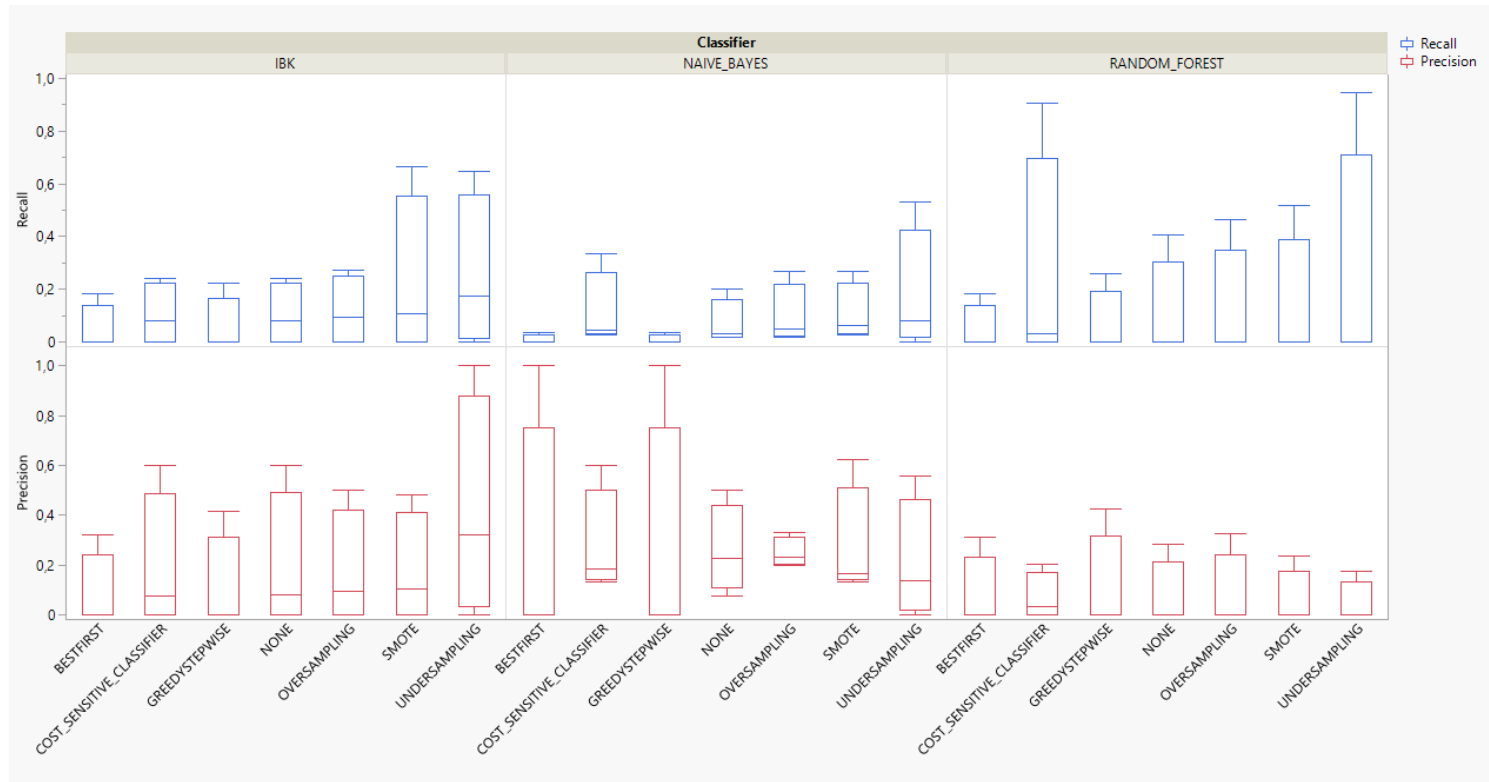
Classificatori e variabili

Sono stati utilizzati tre classificatori, in combinazione con le tecniche elencate qui sotto, per ogni iterazione del Walk Forward:

- Naive Bayes
- Ibk
- Random Forest
- Feature Selection
- Oversampling
- Undersampling
- SMOTE
- Cost Sensitive Learning

Risultati

BookKeeper: Precision, Recall

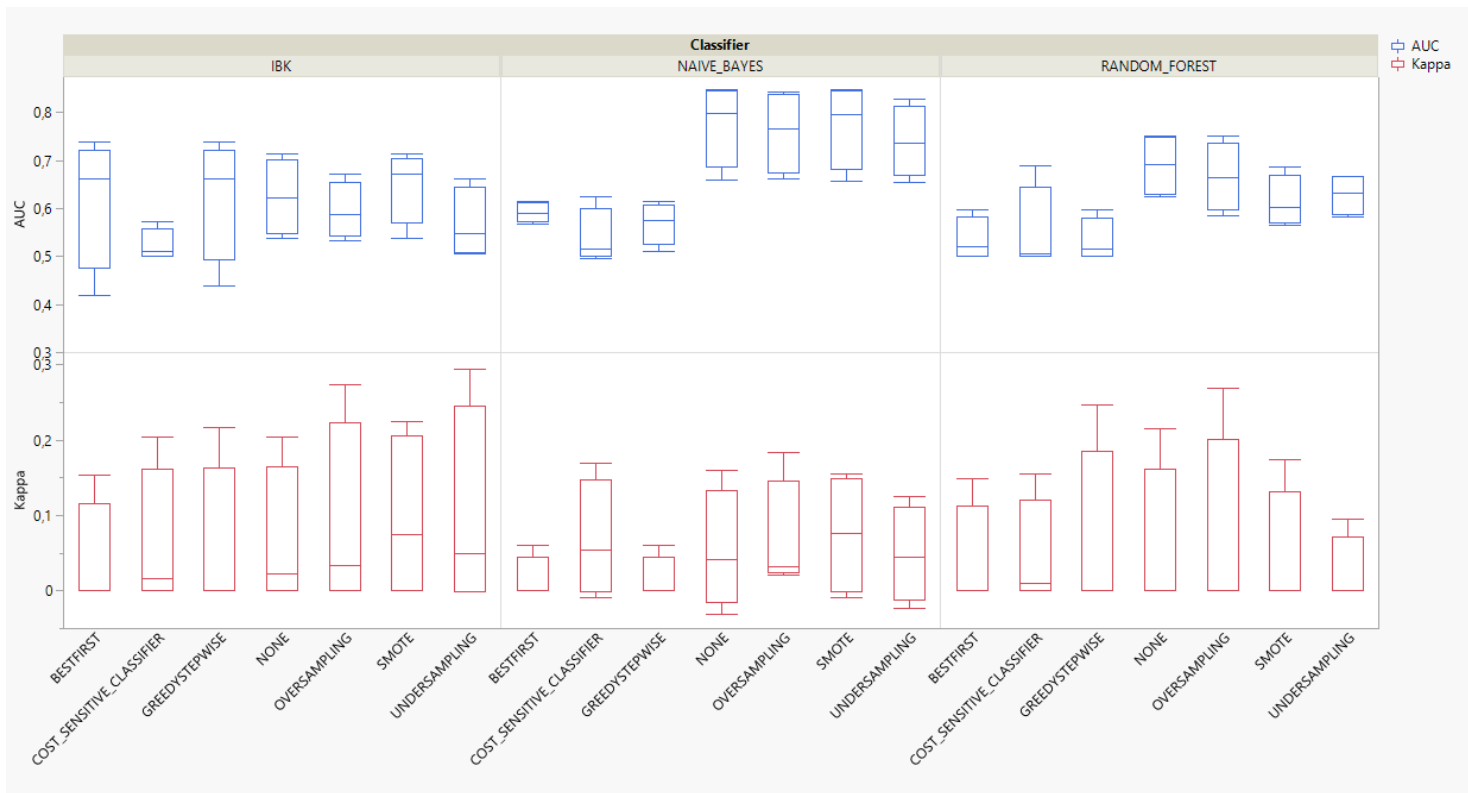


I risultati migliori, in termini di recall vengono raggiunti da **Random Forest**.

Mediamente il più stabile è **IBK**.

Risultati

BookKeeper: Kappa, AUC

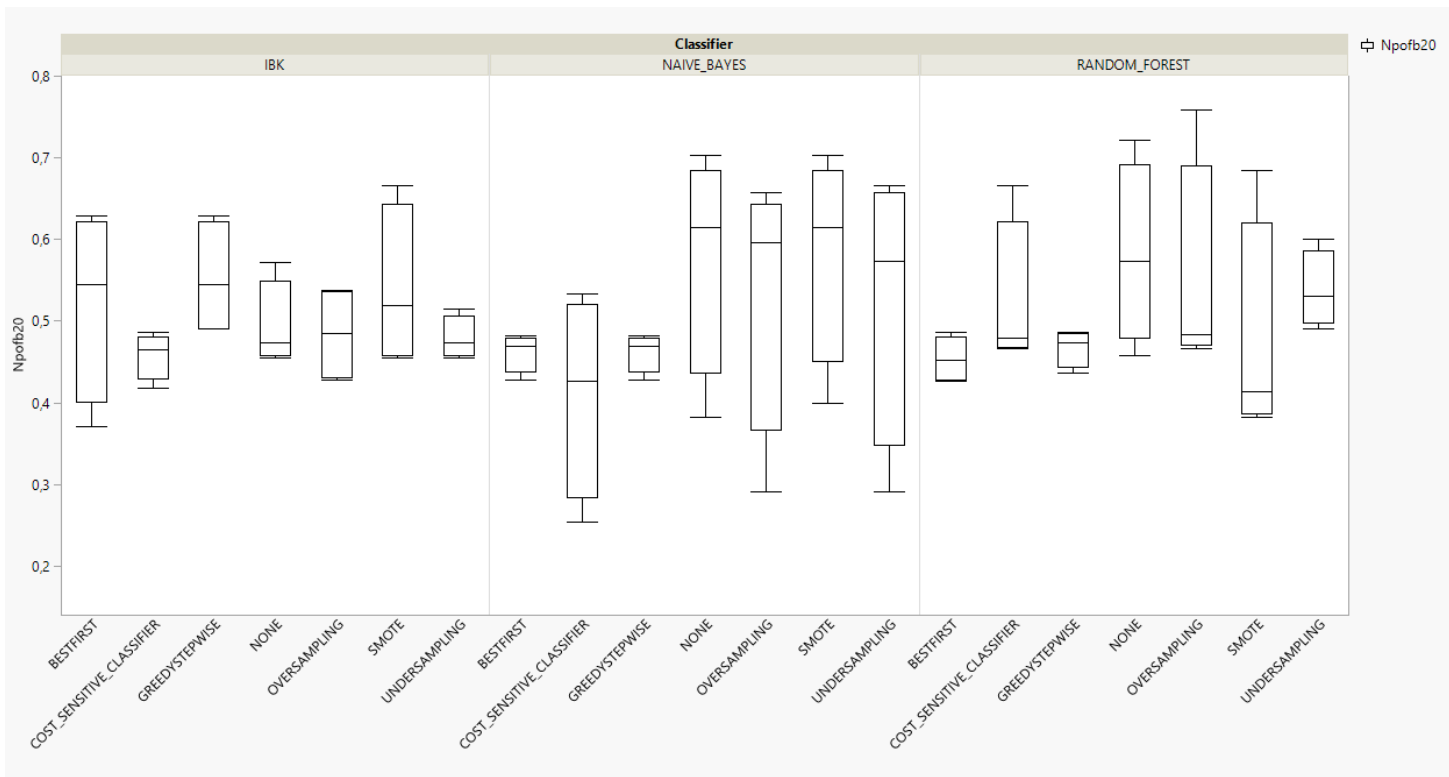


I risultati migliori, in termini di AUC vengono raggiunti da **Naive bayes**.

Non c'è un **classificatore migliore**, che si distingue particolarmente in entrambe le metriche considerate.

Risultati

BookKeeper: NPofB20

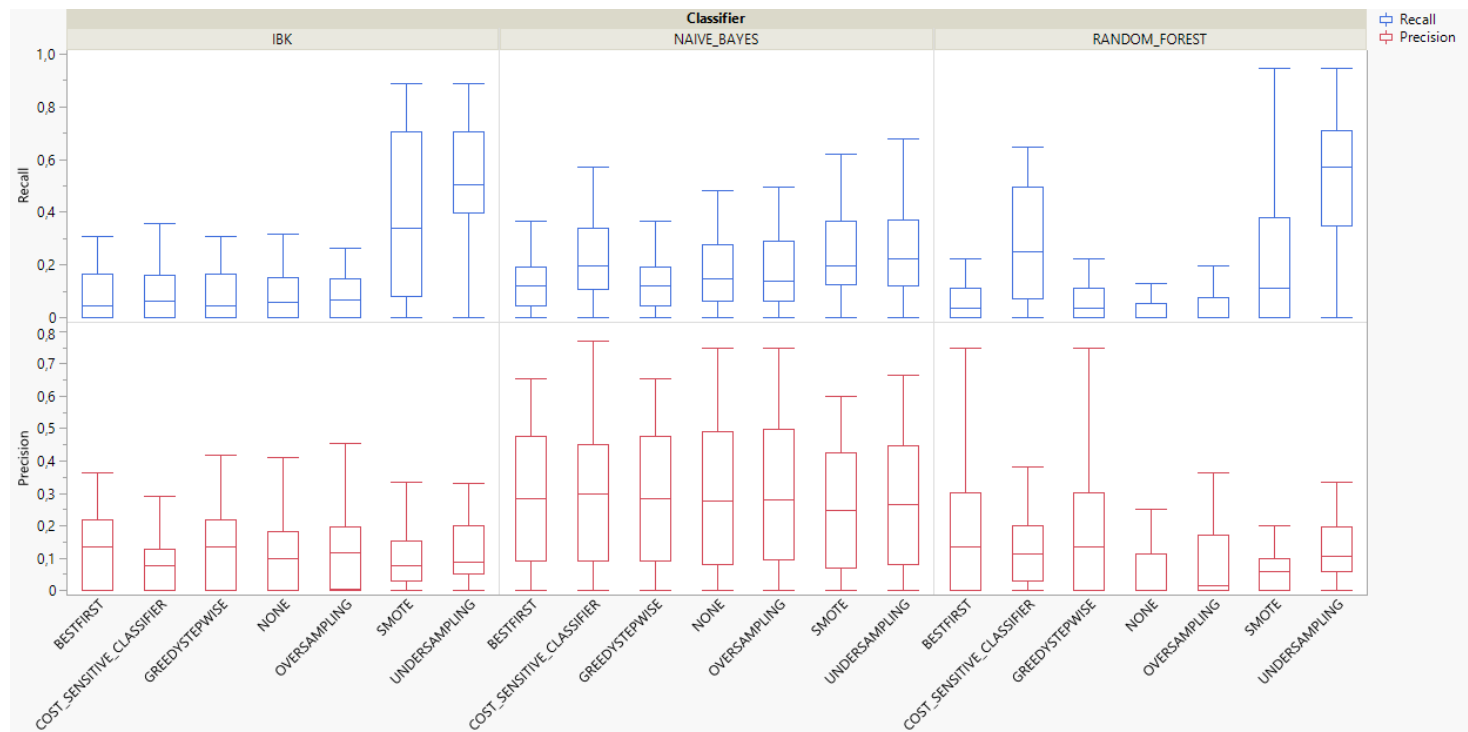


I risultati migliori vengono raggiunti da **Random Forest**.

Tuttavia confrontando le mediane perderebbe quasi sempre il confronto.

Risultati

Syncope: Precision, Recall

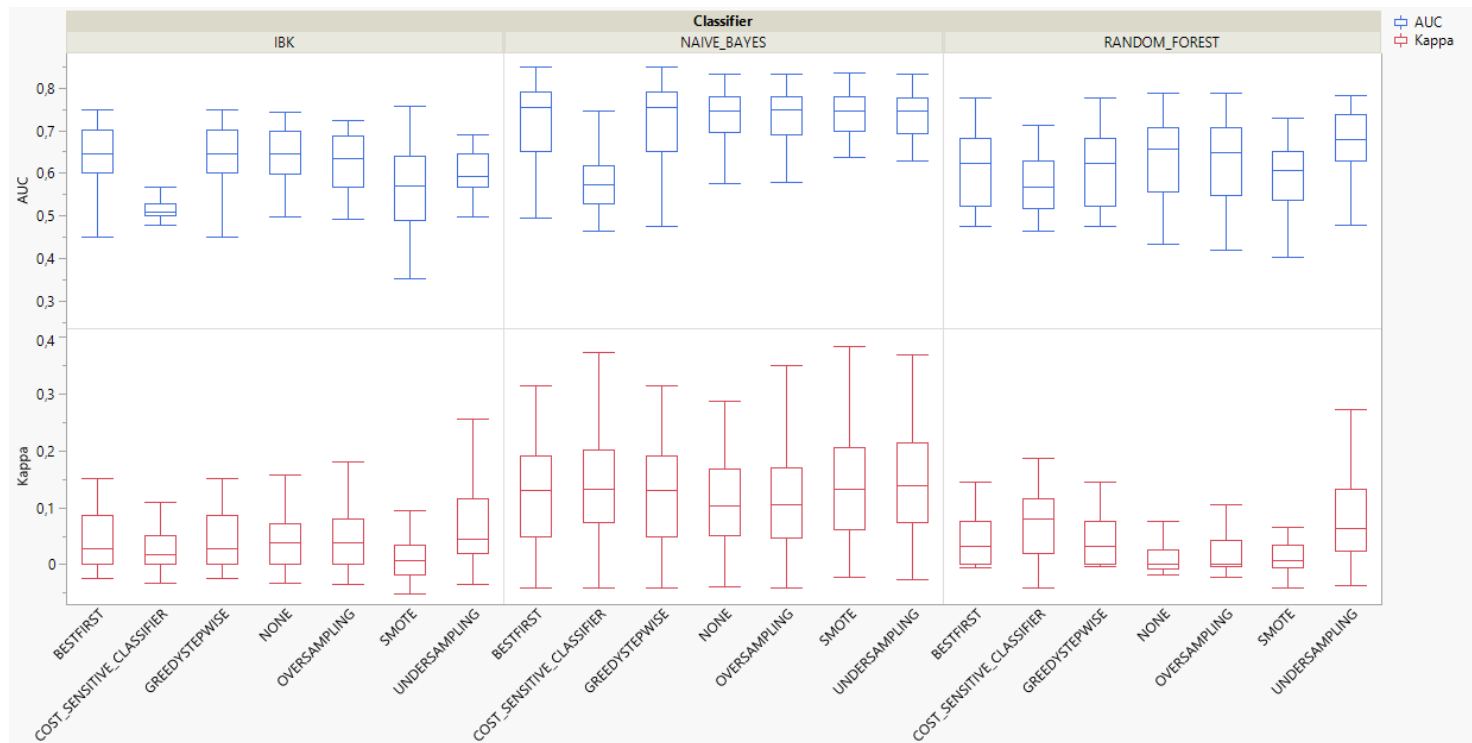


I risultati migliori vengono raggiunti da **Random Forest** in termini di recall.

Non c'è un classificatore che sembra essere il migliore confrontando entrambe le metriche

Risultati

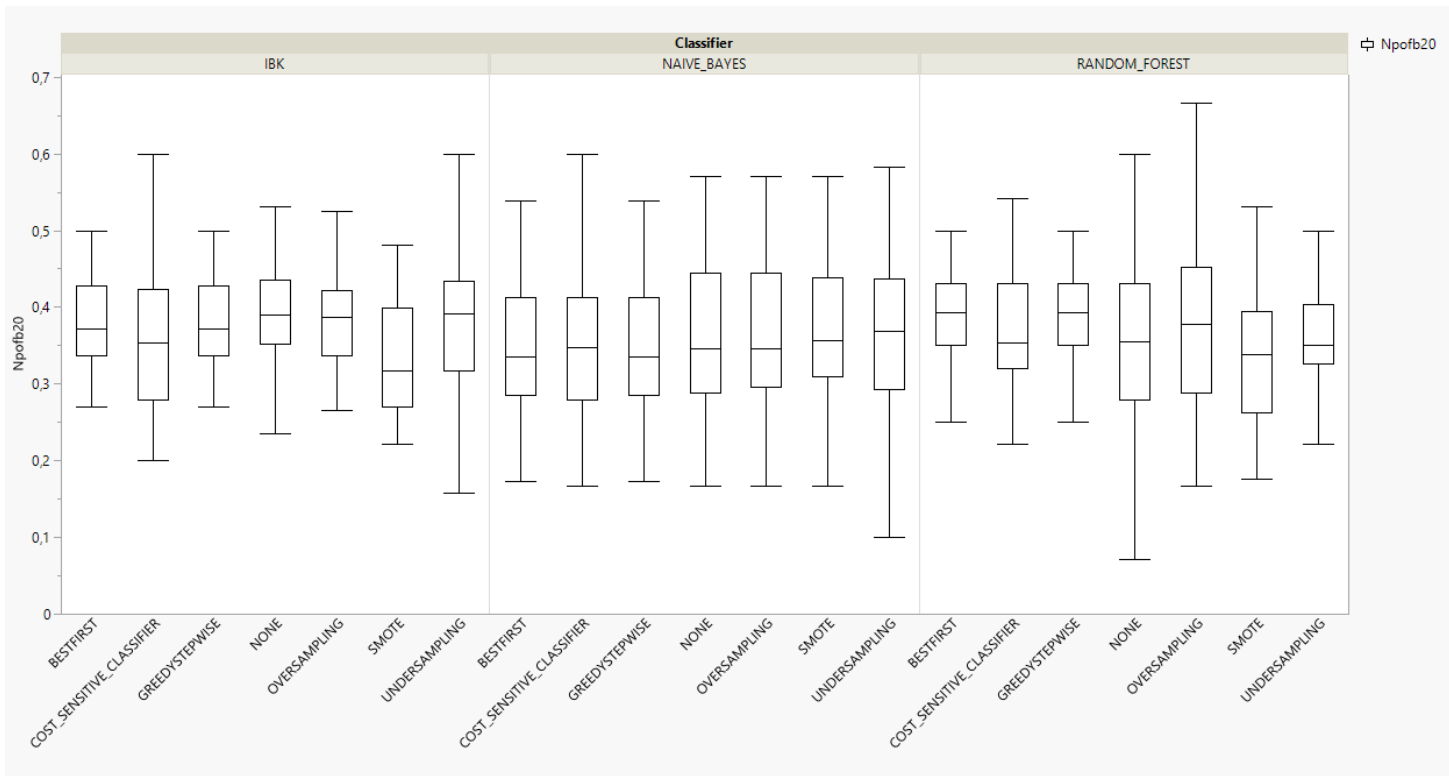
Syncope: Kappa, AUC



Naive Bayes si distingue sia mediamente che come valori massimi in termini sia di AUC che di Kappa

Risultati

Syncope: NPofB20



In termini di NPofB20 non c'è **nessun classificatore** che **si distingue** dagli altri.

Risultati

BookKeeper, Syncope

- **Non c'è un classificatore** nettamente **migliore** degli altri considerando tutte le tecniche che si hanno a disposizione.
- L'unica **tecnica migliore** da usare si è dimostrata essere **l'undersampling**.
- In entrambi i progetti viene selezionata la **metrica personalizzata**.

Syncope:

```
Selected attributes: 1,3,4,11 : 4  
LOC  
NR  
NFix  
AVG_Holiday
```

Bookkeeper:

```
Selected attributes: 4,8,11 : 3  
NFix  
Churn  
AVG_Holiday
```

Minacce alla validità

Cosa può influenzare i risultati?

- I ticket di tipo bug **sono davvero dei bug**?
- Le **tecniche** di proportion, increment e cold start sono **conservative** e potrebbero sottostimare le performance dei classificatori.
- Le **date** riportate su Jira sono accurate?
- I **ticket considerati** per proportion sono davvero validi?

Minacce alla validità

Cosa può influenzare i risultati?

- La scelta dei **progetti** per il calcolo del **cold start**.
- Le **metriche** scelte per la costruzione del dataset.
- La scelta dei **parametri** per il **Cost Sensitive Learning**.
- **Bug nel codice che produce il dataset.**

Link

Reference

Paper Proportion:

Leveraging the Defects Life Cycle to Label Affected Versions and Defective Classes

<https://dl.acm.org/doi/10.1145/3433928>

Paper Snoring:

The Impact of Dormant Defects on Defect Prediction: A Study of 19 Apache Projects.

<https://dl.acm.org/doi/abs/10.1145/3467895>

Link

Progetto



Github:

<https://github.com/EugenioDiGaetano/isw2data>



Sonarcloud:

https://sonarcloud.io/summary/new_code?id=EugenioDiGaetano_isw2data



GRAZIE PER L'ATTENZIONE

Eugenio Di Gaetano - 0349278